Reviewer Pack — Minimal Geometric Entropy Bound on Communication Complexity …

Entry 40b2851a4986 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 12:02:28 UTC

Minimal Geometric Entropy Bound on Communication Complexity Rank Variance

Entry ID: `40b2851a4986` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 12:02:28 UTC

1. Conjecture

Field A (mathematical branch): Geometric Measure Theory (Fractal Dimension) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every boolean function f with n variables and matrix representation M_f, the geometric entropy of the support of M_f is O(f(n)) times the variance in the communication complexity rank among all possible matrix representations of f.

Rationale (proposer's reasoning):

> Geometric entropy captures the information content of a fractal-like structure, which can be used to measure the complexity of a function's representation. By linking this concept to communication complexity, we may expose underlying structural properties that could lead to new lower bounds or separations.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b05171bb2fcc6c4d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for a given boolean function with n variables, the geometric entropy of the support of its matrix representation is within O(f(n)) times the variance in the communication complexity rank among all possible matrix representations, and falsified otherwise.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_representation(f):
        n = len(f.__code__.co_varnames)
        M_f = [[0] * n for _ in range(n)]
        for i in range(2**n):
            inputs = [i >> j & 1 for j in range(n)]
            outputs = f(*inputs)
            if outputs == 1:
                for j in range(n):
                    if inputs[j]:
                        M_f[j][j] += 1
        return M_f

    def geometric_entropy(M):
        n = len(M)
        total = sum(sum(row) for row in M)
        entropy = 0
        for i in range(n):
            p_i = sum(M[i]) / total
            if p_i > 0:
                entropy -= p_i * math.log2(p_i)
        return entropy

    def communication_complexity_rank_variance(f):
        n = len(f.__code__.co_varnames)
        ranks = []
        for i in range(2**n):
            inputs = [i >> j & 1 for j in range(n)]
            outputs = f(*inputs)
            if outputs == 1:
                rank = sum(inputs) + sum(outputs)
                ranks.append(rank)
```

```
        mean_rank = sum(ranks) / len(ranks)
        variance = sum((x - mean_rank)**2 for x in ranks) / len(ranks)
        return variance

    def f_n(x):
        return x[0] and not x[1]

    M_f = matrix_representation(f_n)
    n = len(M_f)
    if n <= 1:
        return {
            "metric_name": "geometric_entropy",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    entropy = geometric_entropy(M_f)
    variance = communication_complexity_rank_variance(f_n)

    if variance == 0:
        return {
            "metric_name": "geometric_entropy",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "variance_is_zero"
        }

    ratio = entropy / variance

    return {
        "metric_name": "geometric_entropy",
        "metric_value": ratio,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True if ratio <= f_n(n) else False,
        "counterexample": "" if ratio <= f_n(n) else "ratio_exceeds_f_n"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results)
```

```
        support_fraction = 1.0
    elif any(not result["conjecture_holds"] and "counterexample" not in result for result in resul
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = next(result["counterexample"] for result in results if "counterexample" i
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(result
    else:
        support_fraction = 0.0

    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fractior
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bafd5c7f.py", line 101, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bafd5c7f.py", line 59, in run_trial
    M_f = matrix_representation(f_n)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bafd5c7f.py", line 26, in matrix_repres
    outputs = f(*inputs)
              ^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bafd5c7f.py", line 57, in f_n
    return x[0] and not x[1]
           ~^^^
TypeError: 'int' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16311
2	preregistration	ollama_remote	glm4:latest	0	0	9271
3	novelty	ollama_remote	glm4:latest	0	0	10693
4	novelty	ollama_remote	glm4:latest	0	0	8416
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13102
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32793
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12841
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10639
9	judge	ollama_remote	glm4:latest	0	0	15401

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 129466 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/40b2851a4986.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/40b2851a4986.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/40b2851a4986.tar.gz` (if generated)